

FavillApp · Architettura

Progetto Flutter (Android) per fumetti episodici. Tutti gli asset (testi, immagini, branch narrativi) sono **pre-generati** e committati nel repo: l'app funziona completamente **offline** per la lettura. Il backend serverless serve solo per le feature opzionali AI e per il loop “chiedi a Favilla reale”.

TL;DR sui due servizi cloud

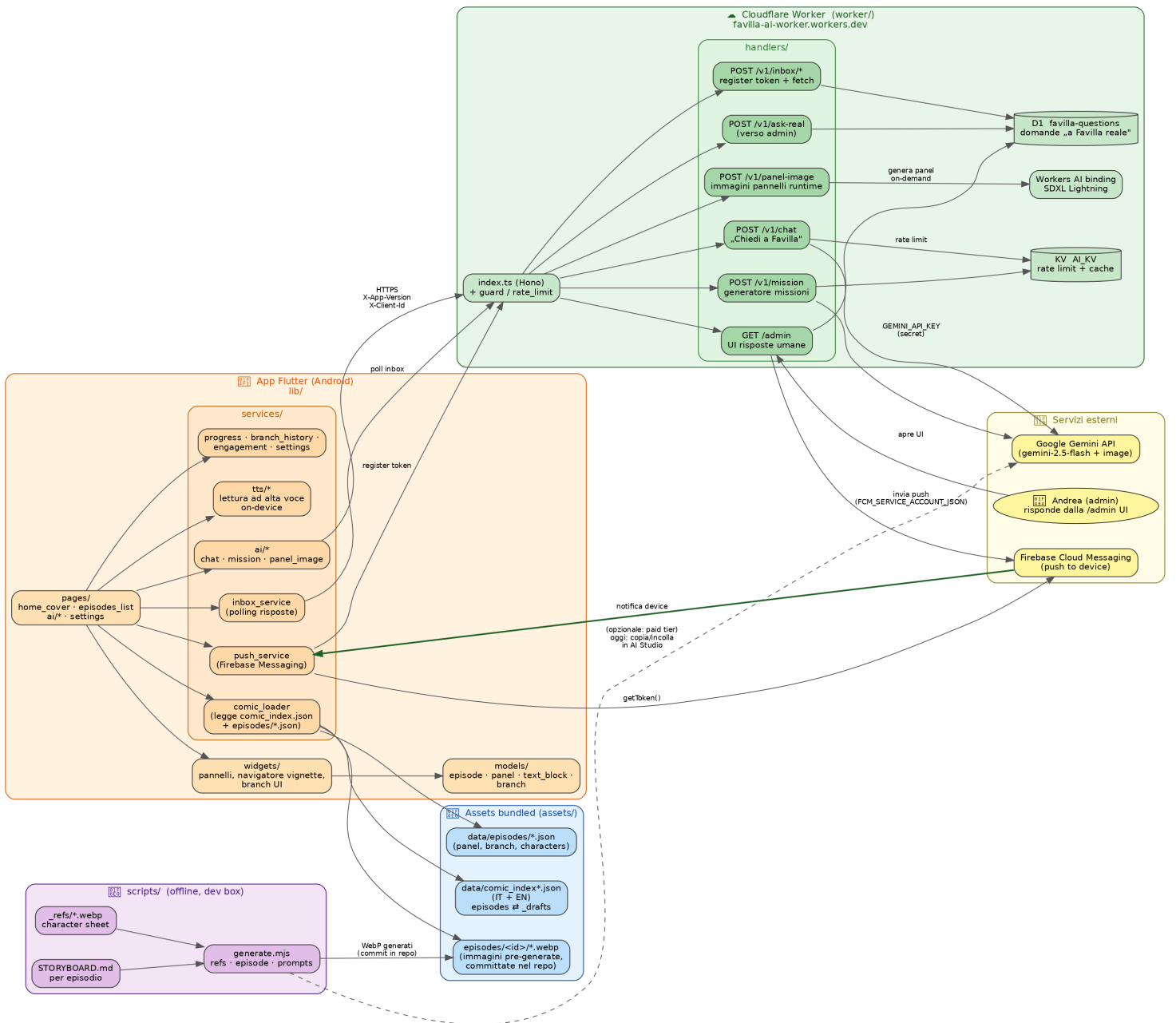
Servizio	A che serve	Cosa NON fa
Cloudflare Worker (<i>worker/</i>)	Proxy HTTP verso Google Gemini. Nasconde la API key (secret Wrangler, mai nell'APK), applica rate-limit e validazione header (KV AI_KV), persiste le domande “a Favilla reale” (D1 favilla-questions), genera le immagini dei pannelli a runtime tramite Workers AI (SDXL Lightning), e invia le push via FCM dalla UI /admin.	Non serve gli asset del fumetto (sono nell'APK). Non sostituisce gli script offline in scripts/.
Firebase	Solo Cloud Messaging (FCM) : notifica push al device quando Andrea risponde da /admin e l'app è chiusa. Il token FCM viene registrato dal device verso il Worker; il Worker firma l'invio con FCM_SERVICE_ACCOUNT_JSON.	Niente Firestore, niente Auth, niente Analytics, niente Crashlytics. Senza FCM l'app funziona lo stesso: l'inbox si sincronizza alla riapertura.

Legenda colori del diagramma

	Blocco	Cosa contiene
	App Flutter (Android)	lib/: pages, widgets, models, services. Gira sul telefono, legge gli asset bundled.
	Assets bundled	assets/: indici episodi, JSON pannelli, WebP. Tutto è dentro l'APK → lettura offline.
	Scripts offline	scripts/: tooling Node che dalla dev box trasforma STORYBOARD.md + character sheet in immagini WebP da committare.
	Cloudflare Worker	Backend serverless edge (Hono). Routes /v1/* per l'app, /admin per Andrea. Storage: KV (rate limit) + D1 (domande).
	Servizi esterni	Google Gemini API, Firebase Cloud Messaging, e Andrea (admin umano).

Note sulle frecce: continue → chiamate / dipendenze attive in produzione; tratteggiate → percorsi opzionali (es. generate.mjs verso Gemini API, oggi disabilitato perché senza billing).

Diagramma dell'architettura



Flussi tipici, passo per passo

1) Lettura di un episodio (offline, niente cloud)

L'utente apre *Episodes List* → `comic_loader` legge `assets/data/comic_index.json` (sezione `episodes`; gli ID in `_drafts`, come **missione_5** oggi, vengono ignorati). Per l'episodio scelto carica `assets/data/episodes/<id>.json` (panel + branch + characters per pannello) e mostra le WebP da `assets/episodes/<id>/`. Il TTS on-device (`services/tts/*`) legge a voce alta. **Nessuna chiamata di rete.**

2) “Chiedi a Favilla” (chat AI → Worker → Gemini)

L'utente scrive in `pages/ai/*`. `services/ai/*` chiama `POST /v1/chat` sul Worker con header `X-App-Version + X-Client-Id`. Il Worker valida (`guard.ts`), incrementa il contatore in KV `AI_KV` (`rate-limit`), costruisce il prompt con il blocco personaggi (`lib/characters.ts`) e chiama Google Gemini usando il secret `GEMINI_API_KEY`. La risposta torna all'app. La key non lascia mai il Worker.

3) “Chiedi a Favilla reale” (loop umano con push)

L'utente invia una domanda “umana” → `POST /v1/ask-real` → il Worker la persiste in D1 (`favilla-questions`). Andrea apre la UI `/admin`, scrive la risposta. Il Worker la salva, e usa `FCM_SERVICE_ACCOUNT_JSON` per inviare una push via Firebase Messaging al token registrato dal device (`services/push_service.dart`). All'apertura della notifica (o al riapri-app) `inbox_service` fa polling di `/v1/inbox/*` e scarica la risposta. Senza FCM funziona lo stesso, ma niente notifica live.

4) Generazione immagini di un nuovo episodio (offline tooling)

Si scrive `assets/episodes/<id>/STORYBOARD.md + assets/data/episodes/<id>.json`. Da `scripts/` si lancia `node generate.mjs episode <id>`: lo script allega le *character sheet* da `scripts/_refs/` e tutte le immagini dell'episodio precedente come *coherence reference*, e chiama Gemini Image. Salva le WebP in `assets/episodes/<id>/`, che si committano. **Stato attuale:** l'API Gemini Image richiede billing; per ora si usa `node generate.mjs prompts <id>` per esportare i prompt pronti da incollare a mano in Google AI Studio (free tier web).

Convenzioni utili

- **Disattivare un episodio:** spostarne l'oggetto da `episodes` a `_drafts` in `comic_index.json` e `comic_index.en.json`. L'app salta automaticamente i drafts.
- **Internazionalizzazione:** ogni asset di testo ha la coppia `foo.json + foo.en.json`. Le immagini sono condivise.
- **Secrets Worker:** `GEMINI_API_KEY`, `ADMIN_TOKEN`, `FCM_SERVICE_ACCOUNT_JSON` via `wrangler secret put`. In locale stanno in `worker/.dev.vars` (gitignored).